

Lista de Exercícios: Árvores Binárias de Busca e Árvores Rubro-Negras*

PCC104 – Projeto e Análise de Algoritmos
DECOM – UFOP

12 de junho de 2026

Instruções. Esta lista cobre **árvores binárias de busca (ABB)** e **árvores rubro-negras (ARN)**. As questões 1 a 16 são teóricas (análise, demonstrações, projeto algorítmico); as questões 17 a 20 envolvem implementação em Python. Em todas as questões, denote por n o número de chaves armazenadas e por h a altura da árvore; nas árvores rubro-negras, adote o nó sentinela $T.nil$ (preto) como folha, no estilo CLRS. Quando solicitada análise assintótica, use notação O , Θ ou Ω , conforme o caso, e justifique a escolha. Marcadores indicam dificuldade: (★) introdutória, (★★) intermediária, (★★★) desafiadora.

Parte I – ABB: Propriedade e Percursos

- (★) (Adaptado de CLRS 12.1-1) Considere o conjunto de chaves $\{1, 4, 5, 10, 16, 17, 21\}$.
 - Enuncie formalmente a **propriedade de árvore binária de busca** e explique por que ela deve valer para *todos* os descendentes de um nó, e não apenas para seus filhos imediatos.
 - Desenhe duas ABBs sobre essas chaves: uma de altura 2 e uma de altura 6.
 - Quantas ABBs *estruturalmente distintas* existem sobre um conjunto de 3 chaves distintas? Desenhe todas elas.
- (★★) **Percurso em ordem.**
 - Apresente o pseudocódigo de INORDER-TREE-WALK e demonstre, por indução no tamanho da subárvore, que ele imprime as chaves em ordem crescente.
 - Mostre que o percurso de uma árvore com n nós leva tempo $\Theta(n)$.
 - (CLRS 12.1-2) Qual a diferença entre a propriedade de ABB e a propriedade de **min-heap**? A propriedade de min-heap permite imprimir as n chaves em ordem crescente em tempo $O(n)$? Justifique usando o limite inferior da ordenação por comparações.
- (★★) (CLRS 12.2-4) Um colega afirma o seguinte: se a busca por uma chave k em uma ABB termina em uma folha, então as chaves da árvore podem ser particionadas em três conjuntos: A (chaves à esquerda do caminho de busca), B (chaves no caminho de busca) e C (chaves à direita do caminho), de modo que $a \leq b \leq c$ para quaisquer $a \in A$, $b \in B$ e $c \in C$. Exiba um **contraexemplo mínimo** para essa afirmação, indicando explicitamente o caminho de busca e os conjuntos A , B e C .

Parte II – ABB: Consultas e Atualizações

- (★) A partir de uma árvore vazia, insira as chaves

41, 22, 64, 10, 30, 55, 70, 28

nessa ordem, usando TREE-INSERT. Mostre a árvore **após cada inserção**. Em seguida, na árvore final, destaque a sequência de nós visitados: (i) na busca bem-sucedida por 28; (ii) na busca mal-sucedida por 56.

- (★★) **Mínimo, máximo e sucessor.**
 - Apresente o pseudocódigo de TREE-MINIMUM e TREE-MAXIMUM e justifique o custo $O(h)$.
 - Apresente TREE-SUCCESSOR, distinguindo os dois casos (x possui ou não filho direito). Justifique a corretude do caso em que x *não* possui filho direito.
 - (CLRS 12.2-5) Mostre que, se um nó de uma ABB tem dois filhos, então seu sucessor não tem filho esquerdo e seu predecessor não tem filho direito.

*Material baseado em Cormen, Leiserson, Rivest e Stein (CLRS), *Introduction to Algorithms*, 4ª ed., MIT Press, 2022, capítulos 12 e 13.

6. (★★) **Remoção.**

- (a) Descreva os três casos de TREE-DELETE (nó sem filhos, com um filho, com dois filhos) e o papel do procedimento auxiliar TRANSPLANT.
- (b) Na árvore final da questão 4, mostre o resultado de remover a chave 22 e, em seguida, a chave 41, indicando em cada remoção qual caso foi aplicado.
- (c) No caso de dois filhos, a chave removida é substituída pelo seu **sucessor**. Justifique por que essa substituição preserva a propriedade de ABB.

Parte III – ABB: Altura e Desempenho

7. (★★) **Tudo depende da altura.**

- (a) Argumente que SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, INSERT e DELETE custam, todas, $O(h)$.
- (b) Quais são a altura **mínima** e a altura **máxima** de uma ABB com n chaves? Exiba, para cada extremo, uma ordem de inserção que o produza.
- (c) Conclua a complexidade de pior caso dessas operações quando as chaves são inseridas em **ordem crescente**.

8. (★★) **Tree sort.** Considere o algoritmo de ordenação que insere as n chaves em uma ABB inicialmente vazia e, em seguida, executa INORDER-TREE-WALK.

- (a) Qual é o custo de pior caso do algoritmo? Que entrada o produz?
- (b) O CLRS mostra que uma ABB **construída aleatoriamente** (chaves inseridas em ordem uniformemente aleatória) tem altura esperada $O(\log n)$. Sob essa hipótese, qual é o custo esperado do *tree sort*?
- (c) Relacione sua resposta com o limite inferior $\Omega(n \log n)$ da ordenação por comparações: o *tree sort* poderia ter custo esperado $o(n \log n)$? Justifique.

Parte IV – Árvores Rubro-Negras: Propriedades

10. (★) **As cinco propriedades.**

- (a) Enuncie as cinco propriedades rubro-negras.
- (b) Defina a **altura negra** $bh(x)$ de um nó x .

11. (★★) **Altura logarítmica (Lema 13.1).**

- (a) Prove, por indução, que a subárvore enraizada em um nó x contém pelo menos $2^{bh(x)} - 1$ nós internos.
- (b) Usando o item (a) e a propriedade que limita os nós vermelhos, conclua que uma árvore rubro-negra com n nós internos tem altura $h \leq 2 \lg(n + 1)$.
- (c) Conclua a complexidade de SEARCH, MINIMUM, MAXIMUM e SUCCESSOR em árvores rubro-negras.

Parte V – Rotações, Inserção e Remoção

13. (★) **Rotações.**

- (a) Apresente o pseudocódigo de LEFT-ROTATE(T, x), indicando quantos ponteiros são atualizados e qual o custo da operação.
- (b) Mostre que a rotação **preserva a propriedade de ABB**. *Sugestão: verifique que o percurso em ordem não se altera.*

14. (★★) (CLRS 13.3-2) A partir de uma árvore rubro-negra inicialmente vazia, insira as chaves

41, 38, 31, 12, 19, 8

nessa ordem, usando RB-INSERT. Mostre a árvore (**com as cores**) após cada inserção, indicando, quando o RB-INSERT-FIXUP for acionado, quais casos foram aplicados.

15. Quais propriedades rubro-negras podem ser violadas imediatamente após a inserção de um nó? Por que o nó inserido é colorido de **vermelho**, e não de preto?

16. (★★★) **Remoção em árvores rubro-negras.**

- (a) Explique o conceito de **preto extra**: por que a propriedade 5 é violada quando o nó removido (ou movido) é preto, e como o “preto extra” depositado sobre um nó x restaura essa propriedade temporariamente, à custa da violação da propriedade 1?
- (b) Descreva os quatro casos do RB-DELETE-FIXUP, em função da cor do irmão w de x e das cores dos filhos de w . Qual caso encerra o laço imediatamente?
- (c) Mostre que ocorrem **no máximo três rotações** e que o custo total do *fixup* é $O(\log n)$. Compare com o RB-INSERT-FIXUP.

Parte VI – Implementação (Python)

Observação. As questões a seguir devem ser entregues no dia da prova.

17. (★★) **ABB básica.** Implemente uma classe `BST` com os procedimentos:

- `TREE-INSERT( $T, z$ )` – inserção em $O(h)$.
- `TREE-SEARCH( $x, k$ )` – em duas versões: **recursiva** e **iterativa**.
- `TREE-MINIMUM`, `TREE-MAXIMUM` e `TREE-SUCCESSOR` – todas em $O(h)$.
- `inorder()` – retorna a lista de chaves em ordem crescente em $\Theta(n)$.

Inclua casos de teste com pelo menos três sequências de inserção distintas, incluindo uma sequência **já ordenada**.

18. (★★) **Remoção com TRANSPLANT.** Estenda a classe `BST` com `TRANSPLANT( $T, u, v$ )` e `TREE-DELETE( $T, z$ )`.

- Seus testes devem cobrir explicitamente os **três casos** de remoção (sem filhos, um filho, dois filhos), incluindo a remoção da raiz.
- Após cada remoção, valide a árvore comparando `inorder()` com a lista esperada.